

**ĐẠI HỌC QUỐC GIA THÀNH PHỐ HỒ
CHÍ MINH**
TRƯỜNG ĐẠI HỌC BÁCH KHOA
**KHOA KHOA HỌC VÀ KỸ THUẬT MÁY
TÍNH**



MÔ HÌNH HÓA TOÁN HỌC
BÀI TẬP LỚN
LỚP L03

STT	Họ tên sinh viên	MSSV
1	Nguyễn Minh Hạnh	2310895
2	Nguyễn Hưng Thịnh	2313287
3	Nguyễn Thái Hoàng	2311062
4	Hoàng Anh Duy	2310458
5	Hồ Công Danh	2310410

BẢNG PHÂN CÔNG CÔNG VIỆC

STT	Họ và Tên	MSSV	Nhiệm vụ chính	Hoàn thành
1	Nguyễn Minh Hạnh	2310895	- Task 2 - Explicit computation of reachable markings - Viết báo cáo	100%
2	Nguyễn Hưng Thịnh	2313287	- Task 1 - Reading Petri nets from PNML files - Viết báo cáo	100%
3	Nguyễn Thái Hoàn	2311062	Task 3 - Symbolic computation of reachable markings by using BDD	100%
4	Hoàng Anh Duy	2310458	Task 5 - Optimization over reachable markings	100%
5	Hồ Công Danh	2310410	Task 4 - Deadlock detection by using ILP and BDD	100%

Mục lục

Tóm tắt (Abstract)	4
1. Giới thiệu	4
2. Lý thuyết nền tảng	4
3. Task 1 — Đọc mạng Petri từ file PNML	5
3.1 Định dạng PNML và Yêu cầu	5
3.2 Thiết kế Cấu trúc Dữ liệu	6
3.3 Thuật toán Hiện thực	8
3.4 Các Chi tiết Hiện thực Quan trọng	10
3.5 Giao diện Public và Cách sử dụng	11
3.6 Kiểm thử và Xác nhận	12
3.7 Tích hợp với các Task sau	13
3.8 Đặc điểm Hiệu năng	13
3.9 Hạn chế và Cải tiến Tương lai	14
3.10 Kết luận	15
4. Task 2 — Phân tích việc hiện thực BFS và DFS	15
5. Task 3 — Tính toán không gian trạng thái bằng BDD	16
6. Task 4	17
1 File: Deadlock_ILP.h	17
1.1 Class DeadlockDetector	17
1.2 Hàm check_reachability_with_bdd	18
2 File: Deadlock_ILP.cpp	18
2.1 Constructor	18
2.2 Hàm get_num_places	18
2.3 Hàm create_dead_marking_ilp_model	18
2.4 Hàm add_dead_marking_constraints	18
2.5 Hàm add_cut_constraint	19
2.6 Hàm find_deadlock_ilp_bdd	19
3 File: main.cpp	19

7. Task 5	19
Các file được thực hiện	20
Độ phức tạp và hiệu năng	22
Hạn chế và hướng cải tiến	22
8. Kết quả thử nghiệm tổng hợp	23
9. Phụ lục	23

Tóm tắt (Abstract)

Bài tập lớn này tập trung vào việc mô hình hoá và phân tích mạng Petri 1-safe bằng ba hướng tiếp cận: (i) phương pháp tường minh (explicit) thông qua liệt kê toàn bộ không gian trạng thái bằng BFS/DFS, (ii) phương pháp biểu tượng dựa trên Binary Decision Diagrams (BDDs) để nén và biểu diễn tập marking một cách hiệu quả, và (iii) sử dụng Integer Linear Programming (ILP) nhằm phát hiện deadlock và thực hiện tối ưu hoá trên tập reachable markings. Mục tiêu của bài tập là giúp sinh viên hiểu rõ bản chất toán học của mạng Petri, nắm được cách triển khai thuật toán trên mô hình rời rạc, và so sánh ưu nhược điểm giữa các phương pháp biểu diễn trạng thái. Các kết quả thu được bao gồm tập reachable markings, phát hiện deadlock nếu có, và lời giải cho bài toán tối ưu tuyến tính trên mạng Petri.

1. Giới thiệu

Mạng Petri là một mô hình hình thức quan trọng trong mô phỏng hệ thống song song, phân tán và theo sự kiện. Nhờ cấu trúc gồm các place, transition và token, mạng Petri cho phép mô tả hành vi hệ thống một cách trực quan nhưng vẫn đảm bảo tính chính xác toán học. Trong thực tế, mạng Petri được ứng dụng trong phân tích giao thức truyền thông, mô hình hoá luồng công việc, hệ thống sản xuất tự động, và kiểm chứng chương trình song song.

Bài tập lớn yêu cầu sinh viên phát triển một hệ thống nhỏ thực hiện ba nhiệm vụ chính: đọc mô hình từ PNML, tính reachable markings bằng hai phương pháp (tường minh và biểu tượng), và phát hiện deadlock bằng ILP kết hợp BDD. Thông qua đó, sinh viên có cơ hội tiếp cận các kỹ thuật thường dùng trong phân tích hình thức như symbolic model checking, representation compression và optimization-based reasoning.

2. Lý thuyết nền tảng

Mạng Petri 1-safe. Một mạng Petri gồm tập place P , tập transition T , ma trận đầu vào I , ma trận đầu ra O , và marking ban đầu M_0 . Với mạng 1-safe, tại mọi thời điểm mỗi place có tối đa một token, giúp giảm kích thước không gian trạng thái và thuận lợi khi mã hoá bằng BDD.

Reachability và đồ thị trạng thái. Reachability là bài toán xác định tất cả marking có thể đạt được từ M_0 thông qua chuỗi firing hợp lệ. Đồ thị reachability có thể trở nên rất lớn do *state-space explosion*, làm nổi bật nhu cầu sử dụng phương pháp symbolic.

Explicit reachability (BFS/DFS). Tiếp cận tường minh duyệt trực tiếp không gian trạng thái bằng BFS hoặc DFS, sinh toàn bộ marking mới từ mỗi marking hiện tại. Phương pháp đơn giản, dễ cài đặt, nhưng không mở rộng tốt khi số marking tăng lớn.

Symbolic reachability bằng BDD. Binary Decision Diagrams (BDDs) cho phép nén tập các marking thành dạng cây quyết định dạng chuẩn (ROBDD), giúp biểu diễn hàng triệu trạng thái mà không cần lưu từng marking tường minh.

Deadlock và ILP. Deadlock là marking trong đó không có transition nào enabled. Kết hợp ILP và BDD cho phép: (i) kiểm tra marking dead có nằm trong reachable set, (ii) phát hiện deadlock với độ chính xác cao nhờ ràng buộc tuyến tính.

Tối ưu hoá tuyến tính trên reachable set. Bài toán yêu cầu tìm marking $M \in Reach(M_0)$ tối đa hoá hàm mục tiêu dạng $c^T M$. ILP hoặc symbolic filtering giúp thu hẹp không gian tìm kiếm, đảm bảo tìm được lời giải hợp lệ hoặc báo không tồn tại nếu reachable set rỗng theo tiêu chí.

3. Task 1 — Đọc mạng Petri từ file PNML

Task 1 tập trung vào việc hiện thực một bộ phân tích cú pháp (parser) mạnh mẽ cho các file PNML (Petri Net Markup Language), chuyển đổi các định nghĩa mạng Petri dựa trên XML thành các cấu trúc dữ liệu nội bộ hiệu quả, phù hợp cho các tác vụ phân tích tiếp theo. Bộ parser phải xử lý được định dạng PNML chuẩn bao gồm các phần tử page lồng nhau, trích xuất tất cả các thành phần mạng và xây dựng các biểu diễn được lập chỉ mục để truy xuất phần tử với độ phức tạp $O(1)$.

3.1 Định dạng PNML và Yêu cầu

PNML là một định dạng chuẩn dựa trên XML để biểu diễn mạng Petri, được sử dụng rộng rãi trong các công cụ kiểm chứng mô hình (model checking). Định dạng này cung cấp một cấu trúc phân cấp với các phần tử chính sau:

- **Places:** Đại diện cho các điều kiện hoặc trạng thái trong hệ thống, mỗi place có một ID duy nhất và tùy chọn marking ban đầu (số lượng token).
- **Transitions:** Đại diện cho các sự kiện hoặc hành động làm thay đổi trạng thái hệ thống.
- **Arcs:** Các kết nối có hướng giữa place và transition (hoặc ngược lại).
- **Pages:** Các container lồng nhau tùy chọn để tổ chức các phần tử mạng theo cấu trúc phân cấp.

Cấu trúc PNML chuẩn tuân theo mẫu sau:

```
<?xml version="1.0" encoding="UTF-8"?>
<pnml xmlns="http://www.pnml.org/version-2009/grammar/pnml">
  <net id="..." type="...">
    <place id="p1">
      <name><text>Place Name</text></name>
      <initialMarking><text>1</text></initialMarking>
    </place>
    <transition id="t1">
      <name><text>Transition Name</text></name>
    </transition>
    <arc id="a1" source="p1" target="t1"/>
    <page id="page1">
      <!-- Nested elements -->
    </page>
  </net>
</pnml>
```

Bộ parser phải xử lý chính xác các phần tử `<page>` lồng nhau, có thể chứa các place, transition, arc hoặc thậm chí các page khác một cách đệ quy.

3.2 Thiết kế Cấu trúc Dữ liệu

Lớp `PetriNet` hiện thực một biểu diễn hiệu quả được tối ưu hóa cho cả hiệu năng lưu trữ và truy xuất. Thiết kế sử dụng các vector được lập chỉ mục kết hợp với hash map để đạt được khả năng truy xuất $O(1)$ trong khi vẫn duy trì khả năng duyệt tuần tự.

3.2.1 Các thành phần dữ liệu cốt lõi

1. Vector lưu trữ ID:

```
std::vector<std::string> place_ids;           // Kích thước: N places  
std::vector<std::string> transition_ids;      // Kích thước: M transitions
```

Các vector này lưu trữ các định danh duy nhất cho tất cả các place và transition. Vị trí chỉ mục đóng vai trò là tham chiếu chính tắc cho phần tử đó trong toàn bộ hệ thống (ví dụ: `place_ids[i]` là "Place i").

2. Vector Marking ban đầu:

```
std::vector<int> initial_marking;             // Kích thước: N places
```

Lưu trữ số lượng token ban đầu cho mỗi place. Đối với mạng Petri 1-safe, các giá trị thường là 0 hoặc 1, mặc dù hiện thực hỗ trợ các giá trị nguyên bất kỳ để tổng quát hóa. Mỗi quan hệ là: `initial_marking[i] = số token trong place_ids[i]`.

3. Ma trận kề (Biểu diễn thưa):

```
std::vector<std::vector<int>> pre_matrix;      // Kích thước: [M] [variable]  
std::vector<std::vector<int>> post_matrix;     // Kích thước: [M] [variable]
```

Các ma trận này sử dụng biểu diễn thưa trong đó:

- `pre_matrix[t]`: Vector các chỉ số place cung cấp đầu vào cho transition `t`.
- `post_matrix[t]`: Vector các chỉ số place nhận đầu ra từ transition `t`.

Ví dụ: Nếu transition 0 có quy tắc $\{p_1, p_3\} \rightarrow \{p_2, p_4\}$, thì:

- `pre_matrix[0] = {1, 3}`
- `post_matrix[0] = {2, 4}`

4. Map truy xuất nhanh (Private):

```
std::unordered_map<std::string, int> place_map;  
std::unordered_map<std::string, int> transition_map;
```

Hash map cho phép chuyển đổi $O(1)$ từ ID dạng chuỗi sang chỉ số nguyên. Các map này được sử dụng nội bộ trong quá trình phân tích cú pháp và được cung cấp thông qua các phương thức getter public.

3.2.2 Ưu điểm của thiết kế này

1. **Chỉ mục hiệu quả:** Chỉ số nguyên cho phép truy cập mảng trực tiếp thay vì so sánh chuỗi.
2. **Lưu trữ cung thừa:** Chỉ lưu trữ các kết nối thực tế, không phải toàn bộ ma trận $N \times M$.
3. **Thân thiện với Cache:** Lưu trữ tuần tự cải thiện việc sử dụng CPU cache trong quá trình duyệt.
4. **An toàn kiểu:** Tách biệt giữa việc tra cứu ID và các thao tác cấu trúc.

3.3 Thuật toán Hiện thực

Việc hiện thực parser sử dụng thư viện TinyXML-2 để xử lý XML và bao gồm hai giai đoạn chính: trích xuất phần tử đệ quy và xây dựng quan hệ cung.

3.3.1 Giai đoạn 1: Trích xuất phần tử đệ quy

Một hàm helper đệ quy `parseElements` duyệt cây XML theo chiều sâu, tự động xử lý các phần tử `<page>` lồng nhau:

```
Input: Phần tử XML, vector lưu trữ, map  
Output: Danh sách place/transition đã điền, danh sách cung thô  
for mỗi phần tử con trong phần tử hiện tại do  
    if con là <page> then  
        | Gọi đệ quy parseElements(con);  
    else if con là <place> với ID hợp lệ then  
        | Thêm ID vào vector place_ids;  
        | Tạo ánh xạ chỉ số trong place_map;  
        | Trích xuất <initialMarking> nếu có;  
        | Lưu số lượng token vào vector initial_marking;  
    else if con là <transition> với ID hợp lệ then  
        | Thêm ID vào vector transition_ids;  
        | Tạo ánh xạ chỉ số trong transition_map;  
    else if con là <arc> với source và target then  
        | Lưu thông tin cung (source, target) vào danh sách tạm;  
end
```

Algorithm 1: Trích xuất phần tử PNML đệ quy

Đệ quy đảm bảo rằng tất cả các phần tử được tìm thấy bất kể độ sâu lồng nhau, giúp parser hoạt động mạnh mẽ với các cấu trúc file PNML khác nhau.

3.3.2 Giai đoạn 2: Xây dựng quan hệ cung

Sau khi tất cả các nút được lập chỉ mục, parser xử lý các cung trong lần duyệt thứ hai để xây dựng `pre_matrix` và `post_matrix`:

Input: Danh sách cung tạm, place_map, transition_map

Output: pre_matrix và post_matrix đã điền

Khởi tạo pre_matrix và post_matrix với các vector rỗng;

for mỗi cung trong danh sách cung tạm **do**

if source là Place VÀ target là Transition **then**

$p_idx \leftarrow \text{place_map}[\text{source}]$;

$t_idx \leftarrow \text{transition_map}[\text{target}]$;

 Thêm p_idx vào $\text{pre_matrix}[t_idx]$;

else if source là Transition VÀ target là Place **then**

$t_idx \leftarrow \text{transition_map}[\text{source}]$;

$p_idx \leftarrow \text{place_map}[\text{target}]$;

 Thêm p_idx vào $\text{post_matrix}[t_idx]$;

end

Algorithm 2: Xây dựng Ma trận Cung

Cách tiếp cận hai giai đoạn này tách biệt các mối quan tâm và đảm bảo tất cả các ID nút đều được biết trước khi cố gắng giải quyết các kết nối cung.

3.3.3 Xác nhận và Xử lý lỗi

Phương thức `verifyConsistency()` thực hiện xác nhận cơ bản:

- Kiểm tra xem có ít nhất một place hoặc transition được tìm thấy hay không.
- Báo cáo thống kê phân tích cú pháp (số lượng place/transition).
- Trả về false nếu mạng rỗng hoặc không đúng định dạng.

Xử lý lỗi bổ sung bao gồm:

- Lỗi tải file (kiểm tra XML_SUCCESS).
- Thiếu các phần tử gốc `<pnml>` hoặc `<net>`.
- Xử lý nhẹ nhàng các thuộc tính tùy chọn bị thiếu (tên, marking).

3.4 Các Chi tiết Hiện thực Quan trọng

3.4.1 Xử lý Page lồng nhau

Việc duyệt đệ quy xử lý chính xác các phần tử `<page>` lồng nhau tùy ý:

```
if (tag == "page") {  
    parseElements(child, p_ids, t_ids, init_marking,  
                  p_map, t_map, raw_arcs);  
}
```

Điều này đảm bảo các mô hình được tổ chức thành các phần logic hoặc phân cấp được phân tích cú pháp chính xác.

3.4.2 Trích xuất Marking ban đầu

Số lượng token được trích xuất từ các phần tử XML lồng nhau:

```
<initialMarking>  
  <text>1</text>  
</initialMarking>
```

Parser điều hướng qua cả hai cấp (`initialMarking` → `text`) và mặc định là 0 nếu phần tử bị thiếu.

3.4.3 Phát hiện hướng cung

Parser tự động xác định hướng cung dựa trên loại nút:

- **Place** → **Transition**: Cung đầu vào, được lưu trong `pre_matrix`.
- **Transition** → **Place**: Cung đầu ra, được lưu trong `post_matrix`.

Việc phát hiện tự động này loại bỏ nhu cầu về các thuộc tính loại cung rõ ràng.

3.5 Giao diện Public và Cách sử dụng

Lớp `PetriNet` cung cấp một giao diện sạch cho các tác vụ phía sau:

Phương thức Phân tích cú pháp chính:

```
bool parsePNML(const std::string& filename);
```

Trả về `true` nếu phân tích thành công, `false` nếu có lỗi.

Phương thức Tra cứu ID:

```
int getPlaceIndex(const std::string& id) const;  
int getTransitionIndex(const std::string& id) const;
```

Trả về chỉ số nguyên cho một ID chuỗi nhất định, hoặc -1 nếu không tìm thấy. Độ phức tạp $O(1)$.

Ví dụ sử dụng:

```
PetriNet net;  
if (net.parsePNML("model.pnml")) {  
    // Truy cập places  
    for (size_t i = 0; i < net.place_ids.size(); ++i) {  
        cout << "Place " << i << ": " << net.place_ids[i]  
            << " (Tokens: " << net.initial_marking[i] << ")" << endl;  
    }  
  
    // Kiểm tra đầu vào transition  
    int t_idx = net.getTransitionIndex("t_produce");  
    for (int p_idx : net.pre_matrix[t_idx]) {  
        cout << "Input: " << net.place_ids[p_idx] << endl;  
    }  
}
```

3.6 Kiểm thử và Xác nhận

Parser đã được kiểm thử trên nhiều loại file PNML:

Danh mục	Số file	Độ phức tạp	Đặc điểm
Example	7	Cơ bản	Các mẫu producer-consumer đơn giản
Simple	29	Thấp	Các mô hình cổ điển (philosophers, mutex)
Medium	7	Trung bình	50-100 places/transitions
Large	8	Cao	200+ places, page lồng nhau

Bảng 1: Các danh mục file kiểm thử PNML

Kết quả kiểm thử:

- Tất cả 51 file kiểm thử đều được phân tích cú pháp thành công.
- Xử lý chính xác các page lồng nhau sâu tới 3 cấp.
- Xác nhận số lượng cung khớp với kiểm tra thủ công.

- Hiệu năng: $< 10\text{ms}$ cho các mô hình đơn giản, $< 500\text{ms}$ cho các mô hình lớn.

Chức năng đã xác minh:

- Trích xuất Place và transition với chỉ mục chính xác.
- Giá trị marking ban đầu khớp với đặc tả PNML.
- Pre-matrix và post-matrix phản ánh chính xác cấu trúc cung.
- Xử lý page lồng nhau bảo toàn tất cả các phần tử.
- Không rò rỉ bộ nhớ (đã xác minh bằng Valgrind trên các bản build Linux).

3.7 Tích hợp với các Task sau

Các cấu trúc dữ liệu đã phân tích hỗ trợ trực tiếp cho tất cả các tác vụ tiếp theo:

- **Task 2 (BFS/DFS):** Sử dụng `initial_marking`, `pre_matrix`, và `post_matrix` để tính toán các transition được kích hoạt (enabled) và bắn (fire) chúng.
- **Task 3 (BDD):** Chuyển đổi chỉ số place thành biến BDD để mã hóa ký hiệu.
- **Task 4 (Deadlock):** Sử dụng pre-matrix để xây dựng các ràng buộc ILP cho các marking chết (dead markings).
- **Task 5 (Optimization):** Hoạt động trên các marking được lập chỉ mục nhất quán với `place_ids`.

Cơ chế lập chỉ mục nhất quán (chỉ số nguyên) loại bỏ chi phí so sánh chuỗi trong các vòng lặp quan trọng về hiệu năng.

3.8 Đặc điểm Hiệu năng

Độ phức tạp thời gian:

- Phân tích cú pháp XML: $O(E)$ với E là số lượng phần tử XML.
- Xử lý cung: $O(A)$ với A là số lượng cung.

- Tổng thể: $O(E + A)$ tuyến tính theo kích thước file.

Độ phức tạp không gian:

- Lưu trữ Place/Transition: $O(N + M)$.
- Lưu trữ cung (thừa): $O(A)$ thay vì ma trận dày $O(N \times M)$.
- Hash map: $O(N + M)$.
- Tổng thể: $O(N + M + A)$ tỷ lệ thuận với kích thước mạng.

Các kỹ thuật tối ưu hóa:

- Biểu diễn ma trận thừa tránh lưu trữ các số 0.
- Hash map cung cấp khả năng tra cứu ID $O(1)$ thay vì tìm kiếm tuyến tính $O(N)$.
- Duyệt đệ quy một lần tránh việc đọc file nhiều lần.
- Dự trữ dung lượng (reserve capacity) trong vector khi biết trước kích thước.

3.9 Hạn chế và Cải tiến Tương lai

Hạn chế hiện tại:

- Giả định trọng số cung là 1 (phù hợp với mạng 1-safe).
- Không xác nhận thuộc tính 1-safe (giả định đầu vào hợp lệ).
- Thông báo lỗi hạn chế đối với XML không đúng định dạng.
- Không hỗ trợ mạng Petri có màu hoặc các phần mở rộng về thời gian.

Cải tiến khả thi:

- Thêm hỗ trợ trọng số cung cho mạng Petri tổng quát.
- Hiện thực xác nhận 1-safe với đầu ra chẩn đoán.
- Cải thiện báo cáo lỗi với số dòng.
- Hỗ trợ các phần mở rộng PNML (màu, thời gian, phân cấp).
- Phân tích cú pháp tăng dần cho các file rất lớn.
- Chuyển đổi đầu ra sang các định dạng khác (DOT, JSON).

3.10 Kết luận

Task 1 đã hiện thực thành công một bộ phân tích cú pháp PNML mạnh mẽ:

- Xử lý chính xác định dạng PNML chuẩn bao gồm các page lồng nhau.
- Cung cấp các cấu trúc dữ liệu hiệu quả được tối ưu hóa cho cả lưu trữ và truy cập.
- Tích hợp liền mạch với tất cả các tác vụ phân tích phía sau.
- Thể hiện hiệu năng tốt trên các mô hình có độ phức tạp khác nhau.

Sự kết hợp giữa TinyXML-2 để xử lý XML và các cấu trúc dữ liệu được thiết kế cẩn thận tạo ra một nền tảng vững chắc cho toàn bộ dự án. Độ tin cậy và hiệu quả của parser cho phép phân tích tự tin các thuộc tính của mạng Petri trong các tác vụ tiếp theo.

4. Task 2 — Phân tích việc hiện thực BFS và DFS

Trong Task 2, hai thuật toán Breadth-First Search (BFS) và Depth-First Search (DFS) được cài đặt trong hai file `BFS.cpp` và `DFS.cpp`. Mục tiêu của hai file này là xác định tập các marking reachable của một mạng Petri 1-safe dựa trên marking ban đầu M_0 , ma trận Input I và Output O .

4.1 Cấu trúc chung trong `BFS.cpp` và `DFS.cpp`

Cả hai file đều triển khai các thành phần chính sau:

- Hàm `_infer_IO`: chuẩn hóa kích thước ma trận I và O về dạng $places \times transitions$.
- Hàm `enabled(M, j)`: xác định transition j có được kích hoạt tại marking M hay không:

$$M[i] \geq I[i][j], \quad M[i] - I[i][j] + O[i][j] \leq 1.$$

- Hàm `fire(M, j)`: tạo marking mới:

$$M_{\text{new}}[i] = M[i] - I[i][j] + O[i][j].$$

- Tập `visited` để tránh thăm lại marking.

4.2 Thuật toán trong BFS.cpp

Sử dụng hàng đợi (queue), khám phá theo mức, đảm bảo tìm được đường ngắn nhất về số bước firing.

4.3 Thuật toán trong DFS.cpp

Sử dụng ngăn xếp (stack), đi sâu theo từng nhánh trước.

4.4 So sánh nhanh giữa BFS.cpp và DFS.cpp

- Cùng trả về tập reachable marking giống nhau.
- BFS dùng queue, DFS dùng stack.
- BFS tìm theo mức, DFS tìm theo chiều sâu.

5. Task 3 — Tính toán không gian trạng thái bằng BDD

Trong Task 3, nhóm chúng em sử dụng kỹ thuật *Symbolic Model Checking* với thư viện **CUDD** để khắc phục hạn chế về bộ nhớ của phương pháp duyệt tường minh khi không gian trạng thái bùng nổ.

5.1 Mã hóa trạng thái và Biến BDD

Để biểu diễn mạng Petri 1-safe, nhóm chúng em ánh xạ mỗi place p_i thành cặp biến Boolean:

- x_i : Trạng thái hiện tại.
- y_i : Trạng thái kế tiếp.

Trong class `BDDReachability`, các biến được khởi tạo xen kẽ theo thứ tự $x_0, y_0, x_1, y_1, \dots$ để tối ưu hóa kích thước biểu diễn của BDD khi các place không thay đổi trạng thái ($x_i \leftrightarrow y_i$).

5.2 Xây dựng Quan hệ Chuyển trạng thái (Transition Relation)

Quan hệ chuyển đổi toàn cục $R(x, y)$ là tổng tuyển (OR) của các quan hệ chuyển đổi riêng lẻ cho từng transition t_k . Với mỗi t_k , logic được xây dựng như sau:

$$R_k(x, y) = \underbrace{\left(\bigwedge_{p \in \bullet t_k} x_p \right)}_{\text{Enabled}} \wedge \underbrace{(\text{Logic cập nhật } y \text{ theo } x)}_{\text{Change}}$$

Phần *Change* xác định trạng thái y dựa trên cấu trúc cung: tiêu thụ token ($y = 0$), sinh token ($y = 1$), hoặc giữ nguyên ($x \leftrightarrow y$) nếu place không liên quan.

5.3 Thuật toán Symbolic BFS

Nhóm chúng em áp dụng thuật toán tìm điểm bất động (Fixed-point iteration) để tính tập trạng thái đạt tới được (*Reach*):

1. Khởi tạo *Reach* từ marking ban đầu M_0 .
2. Tại mỗi bước lặp, tính tập trạng thái kế tiếp bằng phép **AndAbstract**:

$$Next_y = \exists x. (Reach(x) \wedge R(x, y))$$

3. Chuyển đổi biến y về x bằng hàm **SwapVariables** và hợp nhất vào tập *Reach*.
4. Lặp lại cho đến khi tập *Reach* không còn thay đổi.

5.4 Kết quả

Hàm `compute_reachable_markings` trả về:

- BDD biểu diễn tập *Reach* (dùng cho Task 4).
- Số lượng trạng thái chính xác được tính bằng hàm **CountMinterm**, cho phép đếm nhanh chóng ngay cả khi không gian trạng thái rất lớn.

6. Task 4

1 File: `Deadlock_ILP.h`

1.1 Class `DeadlockDetector`

Lớp chính thực hiện thuật toán ILP-BDD. Các thành phần chính:

- PetriNet: mô tả mạng Petri
- Cudd: BDD manager
- Reachable BDD: tập reachable markings

1.2 Hàm `check_reachability_with_bdd`

Kiểm tra một marking nhị phân có thuộc reachable set bằng cách tạo cube BDD và giao với reachable BDD.

2 File: `Deadlock_ILP.cpp`

2.1 Constructor

Gán tham chiếu đến PetriNet, BDD manager và Reachable BDD.

2.2 Hàm `get_num_places`

Lấy số lượng places qua kích thước vector `place_ids`.

2.3 Hàm `create_dead_marking_ilp_model`

Tạo mô hình ILP:

- mỗi biến là nhị phân M_i
- hàm mục tiêu bằng 0 (tìm nghiệm khả thi)
- thêm ràng buộc dead marking

2.4 Hàm `add_dead_marking_constraints`

Ràng buộc đảm bảo mỗi transition không thể kích hoạt:

$$\sum_{P_i \in \bullet T_j} M_i \leq |\bullet T_j| - 1$$

2.5 Hàm `add_cut_constraint`

Loại bỏ nghiệm ILP không reachable:

$$\sum_{i:M_i^*=1} M_i - \sum_{i:M_i^*=0} M_i \leq |\{i|M_i^*=1\}| - 1$$

2.6 Hàm `find_deadlock_ilp_bdd`

Vòng lặp chính:

1. Giải ILP để tìm dead marking M^*
2. Kiểm tra reachable bằng BDD
3. Nếu reachable: trả về deadlock
4. Nếu không: thêm cut constraint
5. Nếu ILP không còn nghiệm: không có deadlock

3 File: `main.cpp`

Quy trình chính:

1. Đọc file PNML
2. Parse mạng Petri
3. Tính reachable BDD
4. Khởi tạo DeadlockDetector
5. Tìm deadlock bằng ILP-BDD
6. In kết quả

7. Task 5

Task 5 tập trung vào việc giải bài toán tối ưu hoá tuyến tính trên tập các marking có thể đạt được từ marking ban đầu M_0 :

$$\text{maximize } c^T M \quad \text{với } M \in \text{Reach}(M_0).$$

7.1 Ý tưởng thực hiện

- Dùng thuật toán khám phá trạng thái (BFS) để thu được toàn bộ tập marking có thể đạt được $\text{Reach}(M_0)$
- Duyệt từng marking trong tập thu được và tính giá trị mục tiêu tuyến tính $v(M) = c^T M = \sum_i c_i m_i$.
- Chọn marking có giá trị lớn nhất làm nghiệm M^* . Nếu tập reachable rỗng thì báo "none".

7.2 Cách thực hiện

Thực hiện theo các bước sau:

1. Tính tập reachable: gọi hàm BFS để lấy danh sách hoặc tập các **Marking**.
2. Chuẩn bị vector trọng số c (kiểu số nguyên, nên dùng 64-bit nếu trọng số lớn).
3. Duyệt danh sách marking: với mỗi marking M , tính $v(M) = \sum_i c_i m_i$; giữ lại giá trị và marking tốt nhất.
4. Trả về M^* và $v(M^*)$; nếu danh sách rỗng trả "none".

Lưu ý :

- BFS có thể tốn thời gian/bộ nhớ nếu không gian trạng thái lớn; trong trường hợp đó cần cân nhắc biểu diễn symbolic bằng BDD hoặc giải pháp ILP kết hợp BDD.
- Đảm bảo dùng kiểu số phù hợp để tránh tràn khi cộng các $c_i m_i$.

7.3 Các file được thực hiện:

- `optimization.h`:
 - Mô tả: Khai báo lớp `Optimizer` và các kiểu dữ liệu liên quan.
 - Giao diện chính: constructor nhận `const std::vector<Marking>& markings`, `const std::vector<long long>& c`, và các phương thức `solve()`, `getOptimalValue()` (kiểu `long long`) và `getComputationTimeMs()` (kiểu `double`).

- Kiểu dữ liệu: dùng alias `using Marking = std::vector<int>;` để biểu diễn một marking; dùng 64-bit cho tổng trọng số để tránh tràn.
- Mục đích: tách phần khai báo để các module khác (ví dụ bộ test) dễ bao gồm và gọi.

- `optimization.cpp`:

- Mô tả: Cài đặt lớp `Optimizer` theo chiến lược vét cạn.
- Thuật toán: hàm `solve()` duyệt toàn bộ markings, với mỗi Marking M tính giá trị

$$v(M) = \sum_i c_i \cdot m_i,$$

sử dụng kiểu `long long` cho tổng; giữ marking có giá trị lớn nhất.

- Đo thời gian: dùng `std::chrono::steady_clock` để đo thời gian thực thi phần tối ưu hoá và lưu vào `computationTimeMs`.
- Trả về: marking tối ưu (hoặc một flag/nút trả về nếu danh sách rỗng) và cho phép truy xuất giá trị tối ưu và thời gian.
- Ghi chú về hiệu năng: độ phức tạp tối đa của bước này là $O(K \times p)$ với K là số marking và p số place.

- `main.cpp` (test harness):

- Mô tả: chứa tập test mẫu dùng để kiểm chứng kết quả và đo thời gian. Các test tiêu biểu:
 - * Test chu kỳ (3 places) — kiểm tra tập reachable nhỏ và nghiệm rõ ràng.
 - * Test phức tạp (nhiều transitions, nhiều places) — kiểm tra tính đúng đắn khi có nhiều marking.
 - * Test stress (ví dụ 5 places tạo 32 marking) — đo thời gian toàn bộ pipeline.
 - * Test trọng số âm / hỗn hợp — kiểm tra xử lý trọng số âm.
- Cách gọi: chương trình minh họa cách lấy danh sách marking (gọi `bfs_reachable(...)` từ module `reachable_marking_BFS_DFS`) rồi khởi tạo `Optimizer` và gọi `solve()`.
- Output: in marking tối ưu, giá trị mục tiêu và thời gian tối ưu hoá.

- Makefile:

- Mục tiêu chính: `all` (biên dịch), `clean` (dọn file tạm), `run` (biên dịch rồi chạy test).

```
cd MHH/Optimization
```

```
g++ -std=c++17 main.cpp optimization.cpp ../reachable_marking_BFS_DFS/E
```

```
./optimization.exe
```

Độ phức tạp và hiệu năng

Gọi $K = |\text{Reach}(M_0)|$, $p = |P|$, $t = |T|$:

- Khám phá BFS: $O(K \times t)$ (xét giới hạn thô khi mỗi marking có thể thử nhiều transition).
- Tối ưu (vết cạn): $O(K \times p)$.
- Tổng: $O(K \times (t + p))$; không gian lưu: $O(K \times p)$ để lưu các marking.

Thực nghiệm trên các test mẫu cho thấy phần tính $c^T M$ rất nhanh; phần tốn tài nguyên nhất thường là bước khám phá reachable.

Hạn chế và hướng cải tiến

- Khi K lớn (ví dụ $> 10^6$), phương pháp vết cạn không khả thi; các hướng cải tiến:
 - Dùng BDD (symbolic) để lưu trữ/lọc marking trước khi tính mục tiêu.
 - Kết hợp ILP: mã hoá biến lựa chọn marking và dùng solver (GLPK/C-PLEX) cùng các ràng buộc cắt (cut) để loại nghiệm không reachable (yêu cầu tương tác với BDD hoặc danh sách marking).
 - Thuật toán tìm kiếm có hướng (branch-and-bound, heuristic) để cắt nhánh và giảm số trạng thái cần đánh giá.
- Sử dụng 64-bit integer cho tổng để tránh tràn khi trọng số lớn.

8. Kết quả thử nghiệm tổng hợp

9. Phụ lục